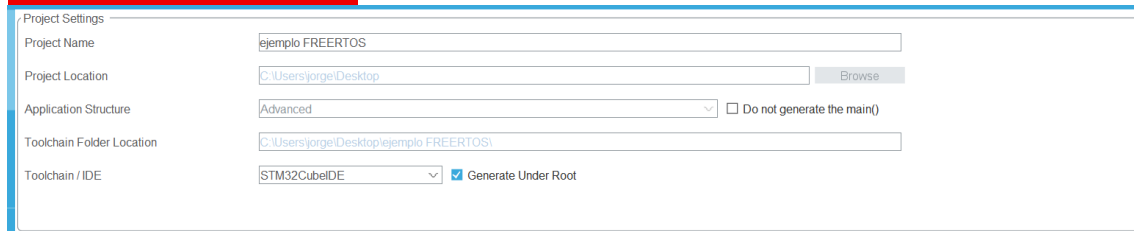


COSAS IMPORTANTES PARA TODOS LOS APARTADOS:



Hay que seleccionar el toolchain de STMCUBEIDE para que se abra el proyecto en el IDE sin problema alguno.

Opción II (SOLO PARA TENERLO EN CUENTA EN EL EJERCICIO III)

El **Filtro de Kalman Extendido (EKF)** es el corazón de la navegación inercial de un cohete: su trabajo es combinar de forma matemática y estadística las lecturas de los sensores (acelerómetros, giróscopos, barómetro, GPS) para estimar en tiempo real la posición, velocidad y orientación exactas del vehículo.

El Filtro de Kalman es un estimador estadístico óptimo porque utiliza una **matriz de covarianza (P)** para saber cuánto debe *confiar* en cada sensor en función de su ruido estimado.

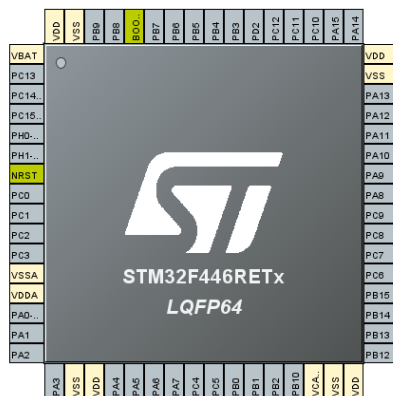
- Las ecuaciones del filtro asumen que el ruido del sensor sigue una distribución normal (gaussiana).
- Cuando un sensor llega a su límite físico y **satura**, deja de medir la realidad y se convierte en un valor plano y constante (el techo del sensor). El modelo estadístico del filtro se rompe porque un valor saturado ya no tiene "ruido normal", tiene un error sistemático gigantesco. El filtro calcula mal su ganancia de Kalman y pierde la capacidad de corregir errores correctamente.

Opción III

Como activar FREERTOS en mi proyecto (con un proyecto ejemplo):

<https://youtu.be/muOL9SH0p9g?is=CQyv1BUL1ZNOinrU>

Usar uno más pequeñito:



USE_PREEMPTION (enabled)

Es el interruptor que activa la planificación preventiva (preemptive scheduling) en el sistema operativo en tiempo real. Para que unas tareas dominen sobre otras. Asegura que las tareas con plazos estrictos (*deadlines* cortos), como leer una IMU o calcular estados de vuelo, no se queden bloqueadas detrás de tareas secundarias que tardan más en ejecutarse.

```
sThreadDef(defaultTask, StartDefaultTask, osPriorityNormal, 0, 128);
```

definir tarea:

osThreadDef(nombre tarea, punto de entrada de la tarea, prioridad de la tarea, stack size)

Retocar el código que viene dado de FreeRTOS para que empiece a funcionar:

1. Como se haría de normal sin RTOS:

bare-metal (o programación secuencial tradicional usando solo la HAL).

Al comentar la línea del kernel, eliminamos el sistema operativo de la ecuación. El microcontrolador se olvida de las tareas, las prioridades y el planificador, y se limita a ejecutar el código de forma estrictamente lineal dentro del bucle principal while(1), usando retardos bloqueantes tradicionales como HAL_Delay().

```
138 while (1)
139 {
140     /* USER CODE END WHILE */
141
142     /* USER CODE BEGIN 3 */
143     HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_0);
144     HAL_Delay(1);
145     HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_1);
146     HAL_Delay(2);
147 }
148 /* USER CODE END 3 */
149 }
```

```
132 //osKernelStart();
```

2. Como se haría de normal con RTOS:

En la tarea 1 ponemos el pin 0 y en la tarea 2 (la que hemos creado) ponemos el pin 1.

USAR LOS BUCLES DE TAREAS:

```
277 void StartDefaultTask(void const * argument)
278 {
279     /* USER CODE BEGIN 5 */
280     /* Infinite loop */
281     for(;;)
282     {
283         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_0);
284         osDelay(1);
285     }
286     /* USER CODE END 5 */
287 }
```

```

296 void Task2_init(void const * argument)
297 {
298     /* USER CODE BEGIN Task2_init */
299     /* Infinite loop */
300     for(;;)
301     {
302         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_1);
303         osDelay(1);
304     }
305     /* USER CODE END Task2_init */
306 }

```

Dos tareas se pueden acometer a la vez incluso con una pequeña diferencia de 1ms.



INFO FREERTOS:

Son archivos de C que se instalan en mi proyecto de CubeIDE. Que permite hacer varios bucles/tareas a la vez.

SCHEDULER (o planificador):

Administra el tiempo de CPU para cada tarea. Determina qué tarea debe estar en ejecución en un momento dado.

Por ejemplo elegir entre las tareas siguientes:

```

void Tarea1(void)
{

```

```

while(1)
{
GPIO_LED3_Toggle();
Delay_ms(500);
}
void Tarea2(void)
{
while(1)
{
GPIO_LED2_Toggle();
Delay_ms(500);
}
}

```

DESPACHADOR: provee el control del CPU a una cierta tarea en el momento adecuado.

TAREAS: tiene su propio while, pero gracias al Schedule se ejecutan.

Tareas cooperativas(cada una tiene su hueco/apropiativas(contexto de ejecución es guardado por el despachador y lo reemplaza por la nueva tarea)

TAREAS SE EJECUTAN EN PARALELO

```

#include "FreeRTOS.h"
#include "task.h"
#include "sensors/sensors.h"

// Definimos la función que ejecutará la tarea del sensor
void vSensorTask(void *pvParameters) {
    // Variable para controlar el periodo exacto de la tarea
    TickType_t xLastWakeTime;
    const TickType_t xFrequency = pdMS_TO_TICKS(10); // 10 ms (100 Hz)

    // Inicializamos el contador de tiempo con el instante actual
    xLastWakeTime = xTaskGetTickCount();

    for (;;) {
        // 1. Leemos los sensores (o el sensor específico)
        sensors_snapshot_t snapshot;
        // sensors_read_snapshot(&snapshot); // (Ejemplo de función de lectura)

        // 2. Hacemos algo con los datos (ej: pasarlos al filtro de Kalman o guardar en cola)
        // procesar_datos(&snapshot);

        // 3. Esperar exactamente hasta el siguiente ciclo (NO bloquea el procesador)
        // FreeRTOS duerme esta tarea y deja libre la CPU para otras tareas urgentes.
        vTaskDelayUntil(&xLastWakeTime, xFrequency);
    }
}

```

vTaskDelayUntil frente a HAL_Delay: Mientras que HAL_Delay(10) mantiene al procesador atrapado en un bucle vacío gastando ciclos y tiempo de CPU, vTaskDelayUntil le dice al sistema operativo: *"Oye, a esta tarea no la despiertes hasta dentro de exactamente 10 milisegundos"*.

Liberación de la CPU: Durante ese tiempo de espera, FreeRTOS aprovecha para ejecutar otras tareas (como la telemetría, el control de los motores o el registro de datos en la tarjeta Flash).

Determinismo: Las tareas de mayor prioridad (como la lectura de la IMU mediante la interrupción del pin físico) pueden interrumpir a las de menor prioridad al instante, garantizando que el cohete o dron reaccione con precisión milimétrica.

APLICAR freeRTOS a mi programa:

Conceptos a tener en cuenta...

Tarea (Thread): Es como un mini-programa independiente dentro de tu microcontrolador. Cada tarea tiene su propia función con su bucle infinito **for(;;)**. El planificador (*scheduler*) decide a cuál le da un ratito de CPU.

Prioridad (y Preemption): Como vimos antes con el parámetro USE_PREEMPTION, si tienes dos tareas listas para correr, el sistema ejecuta la que tenga mayor prioridad. Si entra una tarea muy urgente (como leer la IMU), interrumpe a la que estaba corriendo y toma el control inmediatamente.

Mutex (Mutually Exclusive / Exclusión Mutua): El mutex hace que una tarea pida la “llave” del bus I2C, haga su lectura y la devuelva para que otro la use.

Cola (queue): Cola de espera tipo FIFO: First In, First Out, el primero que llega es el primero que se atiende).

Thread Flag (semáforo binario): A diferencia de la cola (que transporta datos), esto es una señal de aviso sin datos. “interrupción”.

Event flags (bandera de eventos): Son como un panel con varios interruptores. Permiten que varias tareas sepan en qué fase de vuelo está el cohete (despegue, motor encendido, apogeo, caída ...)

“osDelay” y “osDelayUntil” (para tareas periódica)// HAL_Delay (bloquea CPU)

*****USE_PREEMPTION` (enabled):***

****osThreadDef(nombre, función, prioridad, pila):**** es la API ****CMSIS_V1****

****CMSIS_V2**→osThreadNew()**

****Tu `vTaskDelayUntil(&xLastWakeTime, xFrequency)\*\*** es la versión nativa de FreeRTOS de `osDelayUntil()

****Lo que cambia respecto a tu ejemplo:**** en `vSensorTask` lees **todos** los sensores en una sola tarea a 100 Hz. En la aviónica no vale: la IMU necesita 1 kHz y el barómetro tarda 10 ms por conversión. Por eso se separa una tarea por ritmo (sección 2), y cada una se comunica con las demás por colas (sección 3).

El ESQUEMA DE LO QUE QUEREMOS DESARROLLAR:

1. Configuración en CubeMX (abre `FARADAYJorge\_Ej3.ioc`)

1. ****System Core → SYS → Timebase Source → TIM6.****

- Por qué: FreeRTOS usa SysTick para su reloj. Si la HAL también lo usa, chocan; CubeMX te avisa.

2. ****Middleware → FREERTOS → Interface: CMSIS_V2.****

3. ****Config parameters:****

- `TOTAL\_HEAP\_SIZE`: súbelo, por ejemplo a 32 768 bytes (tienes 1 MB de RAM).
- `USE\_PREEMPTION`: Enabled (viene por defecto).
- `CHECK\_FOR\_STACK\_OVERFLOW`: opción 2, para detectar desbordamientos de pila (robustez).

4. ****Tasks and Queues:**** crea las tareas de la tabla de la sección 2 (nombre, prioridad, pila, **entry function**).

- **Entry function** = el nombre de la función, por ejemplo `fTareaIMU`.
- Pila: 512 palabras para las normales y 1024 para el filtro (usa matrices).

Tarea	Función	Frecuencia	Prioridad CMSIS	Por qué esa prioridad
tIMU	fTareaIMU	1 kHz, despertada por la interrupción de dato listo (PE6)	osPriorityRealtime	El plazo más corto del sistema: si pierde una muestra, la integración inercial se degrada. Su trabajo es corto (una lectura SPI de 14 bytes).
tAltaG	fTareaAltaG	1 kHz durante la propulsión; 100 Hz después	osPriorityHigh7	Solo importa en el empuje, donde la IMU satura. Mismo ritmo que la IMU, pero menos crítica fuera del empuje.
tEventos	fTareaEventos	100 Hz	osPriorityHigh	Detecta despegue, apagado y apogeo, y dispara las cargas de recuperación . Llegar tarde al apogeo = paracaídas tarde = pérdida del cohete.
tEstimacion	fTareaEstimacion	200 Hz	osPriorityAboveNormal	El filtro de Kalman. Es lo que más CPU gasta (~16 Mciclos/s); si fuera más alta, robaría tiempo a la IMU y a los eventos.

Tarea	Función	Frecuencia	Prioridad CMSIS	Por qué esa prioridad
tBaro	fTareaBaro	50 Hz (máquina de estados: pedir conversión → esperar → leer)	osPriorityNormal	Lento por naturaleza (10 ms por conversión). Mientras espera duerme con osDelay, sin bloquear. Deja de servir por encima de ~30 km.
tMag	fTareaMag	100 Hz	osPriorityNormal	Corrección de guiñada: lenta y tolerante a retrasos.
tGNSS	fTareaGNSS	10 Hz (frase NMEA o UBX por UART con DMA)	osPriorityNormal	Ayuda intermitente: se bloquea en el ascenso (COCOM). Que llegue tarde no rompe nada.
tRegistro	fTareaRegistro	200 Hz (vacía la cola a la flash NOR por bloques)	osPriorityBelowNormal	Que no se pierdan datos importa, pero la cola absorbe los retrasos. Es la que más tarda (escritura en flash), así que no puede estar por encima del control.
tTelemetria	fTareaTelemetria	10 Hz	osPriorityLow	Útil para tierra, nada en vuelo depende de ella.
tAerofreno	fTareaAerofreno	50 Hz, por CAN	osPriorityBelowNormal	Control de apogeo: necesita la estimación, pero un retraso de unos ms no es crítico.
tSalud	fTareaSalud	1 Hz	osPriorityLow	Batería (INA226), temperaturas y estado de los sensores. Refresca el watchdog (IWDG) solo si todas las tareas críticas han dado señales de vida.
tZumbador	fTareaZumbador	según el evento	osPriorityIdle + 1	Avisos sonoros en rampa y en la búsqueda. Lo último en importancia.

Mode

Interface CMSIS_V2

Configuration

Reset Configuration

Tasks and Queues

Timers and Semaphores

Mutexes

Events

FreeRTOS Heap Usage

Config parameters

Include parameters

Advanced settings

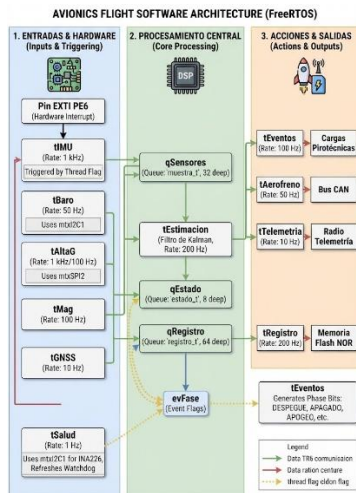
User Constants

Tasks

Task Name	Priority	Stack Size (Wor	Entry Function	Code Generatio	Parameter	Allocation	Buffer Name	Control Block Na
Task2	osPriorityNormal	128	Task2_init	Default	NULL	Dynamic	NULL	NULL
tIMU	osPriorityRealti...	512	fTareaIMU	Default	NULL	Dynamic	NULL	NULL
tAltaG	osPriorityHigh7	512	fTareaAltaG	Default	NULL	Dynamic	NULL	NULL
tEventos	osPriorityHigh	512	fTareaEventos	Default	NULL	Dynamic	NULL	NULL
myTask05	osPriorityAbove...	1024	fTareaEstimacion	Default	NULL	Dynamic	NULL	NULL
tBaro	osPriorityNormal	512	fTareaBaro	Default	NULL	Dynamic	NULL	NULL
tMag	osPriorityNormal	512	fTareaMag	Default	NULL	Dynamic	NULL	NULL

AddDelete

5. ****Colas, mutex y event flags**** (sección 3). Se crean en la misma pantalla, en las pestañas **Queues**, **Mutexes** y **Events**.



1. Bloque de Entradas y Sensores (Adquisición)

- **La regla de oro de la IMU (tIMU a 1 kHz):** El pin de hardware PE6 genera una interrupción cuando hay un dato listo. Como regla de diseño, *la interrupción no hace trabajo pesado*: solo activa un *Thread Flag* para despertar instantáneamente a la tarea tIMU y así evitar la degradación de la navegación inercial.
- **Gestión de buses con Mutexes:** Sensores como el barómetro (tBaro), el sensor de salud (tSalud con el INA226) o el acelerómetro de alta aceleración (tAltaG) comparten buses físicos (I2C o SPI). Para evitar corrupción de datos y la inversión de prioridad, utilizamos **Mutexes** con herencia de prioridad.

2. Bloque de Procesamiento Central (El Núcleo del Sistema)

- **El embudo de datos (qSensores):** Todas las tareas de sensores vuelcan sus lecturas en una cola tipo FIFO (muestra_t de 32 elementos). Esto actúa como un búfer que absorbe picos de tráfico antes de que lleguen al procesador.
- **El Filtro de Kalman (tEstimacion a 200 Hz):** Es el corazón matemático. Procesa los datos y genera un estado unificado (qEstado) que distribuye la altitud y velocidad estimada.
 - *Tip de entrevista:* Aunque el filtro consume mucha CPU, **no tiene la máxima prioridad** deliberadamente; si la tuviera, robaría ciclos vitales a la IMU. Su prioridad es intermedia (*Above Normal*).
- **Fases de vuelo (evFase):** Mediante *Event Flags*, se comunican los cambios críticos de estado (despegue, apagado, apogeo) a todo el software para que cada tarea adapte su comportamiento.

3. Bloque de Acciones y Salidas (Seguridad y Telemetría)

- **Tareas críticas de seguridad (tEventos y tAerofreno):** Monitorean continuamente el apogeo y los tiempos de la misión. Fallar aquí o llegar tarde significa perder el cohete, por lo que tienen alta prioridad.
 - **La Caja Negra (tRegistro a 200 Hz):** Todas las tareas escriben sus trazas en una cola grande (qRegistro de 64 elementos). Una tarea de baja prioridad se encarga de vaciarla poco a poco escribiendo por bloques en la memoria Flash NOR externa, asegurando que la escritura lenta no bloquee el control del cohete.
1. **Prioridades basadas en consecuencias:** "Asigno prioridades según el plazo y la consecuencia de fallar: arriba lo más urgente y crítico para salvar la misión (IMU y eventos); abajo lo que solo sirve en tierra (telemetría y zumbador)".
 2. **Uso de osDelayUntil:** "Ninguna tarea periódica usa delays bloqueantes tontos (HAL_Delay), todas usan retardos absolutos para evitar la acumulación de deriva temporal".
 3. **Salud y Watchdog:** "La tarea de salud (tSalud a 1 Hz) verifica los voltajes y temperaturas, y solo refresca el watchdog del hardware si todas las tareas críticas han demostrado que siguen vivas y ejecutándose correctamente".

Una vez tengas rellenas estas pestañas con sus nombres exactos, al hacer clic en **GENERATE CODE**, CubeMX creará automáticamente las variables globales de las colas y mutexes (como osMessageQueueId_t qSensoresHandle; y osMutexId_t mtxI2C1Handle;) listas para que las uses en tus tareas.

Como se almacenan las estructuras actuales:

```
typedef struct {
    uint32_t timestamp;
    float ax, ay, az; // Aceleraciones
    float gx, gy, gz; // Girascopos
    float presion;
} muestra_t;
```

6. ****NVIC:**** las interrupciones que llamen a funciones de FreeRTOS tienen que tener prioridad numérica ****≥ 5**** (configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY). Pon las EXTI de los sensores en 5 o 6.

7. ****Generate Code.****

- CubeMX crea `Core/Src/app\_freertos.c` (o mete todo en `main.c`) con un esqueleto vacío por tarea.
- Tú escribes ****dentro de `USER CODE BEGIN fNombre`****, que es lo que se conserva al regenerar.

> Al compilar, el `HAL\_Delay` y el `printf` del bucle de `main.c` dejan de ejecutarse: con FreeRTOS, `main()` llama a `osKernelStart()` y no vuelve. Mueve la lectura de sensores a sus tareas.

- TENER CUIDADO CON:
Además, al abrir la carpeta del proyecto directamente como un *workspace* de Eclipse/CubeIDE, el entorno a veces se descoloca o carga referencias antiguas. Por eso se recomienda abrir tu espacio de trabajo habitual e importar el proyecto si hace falta.

COMO SE ESTA IMPLEMENTANDO EL CÓDIGO:

Paso A: define el *flag* de la IMU

En main.c, busca `/* USER CODE BEGIN PD */` (línea 56) y escribe debajo:

```
#define FLAG_IMU_DRDY 0x0001U /* aviso de la ISR de PE6 a tIMU: «la IMU tiene dato nuevo» */
```

Ctrl+S, Ctrl+B.

Paso B: la interrupción solo avisa

Busca `/* USER CODE BEGIN 4 */` (línea 1433) y escribe debajo:

`/* La ISR de las EXTI. Regla: una interrupción no hace trabajo, solo avisa.`

`* La lectura SPI la hace tIMU, que es una tarea y puede esperar al bus. */`

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
```

```
{
    if (GPIO_Pin == interrupciones_sensores_Pin) /* PE6: dato listo de la IMU */
    {
        osThreadFlagsSet(tIMUHandle, FLAG_IMU_DRDY);
    }
}
```

Ctrl+B.

Comprobación en CubeMX: abre el .ioc y ve a **System Core** → **NVIC**. La línea **EXTI line[9:5]** tiene que tener una *Preemption Priority* de **5 o más**. Si está en 0, ponla a 5 y guarda. Una interrupción con prioridad más alta que 5 no puede llamar a funciones de FreeRTOS: el micro se colgaría.

Paso C: el bucle real de tIMU

En fTareaIMU, debajo de tu comentario, sustituye el `for(;;) { osDelay(1); }` que puso CubeMX por esto:

```
muestra_t m;
icm42686_data_t imu;
for (;;)
{
    /* Duermo hasta que la ISR avise (máximo 5 ms) */
    uint32_t f = osThreadFlagsWait(FLAG_IMU_DRDY, osFlagsWaitAny, 5);
    if (f & osFlagsError)
    {
        continue; /* timeout: la IMU no ha avisado (avisar a tSalud) */
    }
    if (icm42686_read(&imu) == SENS_OK)
    {
        m.t_us = osKernelGetTickCount() * 1000U; /* ms -> µs (luego, con TIM2) */
        m.sensor = 0; /* 0 = IMU */
        m.valor[0] = imu.ax_g;
        m.valor[1] = imu.ay_g;
        m.valor[2] = imu.az_g;
        osMessageQueuePut(qSensoresHandle, &m, 0, 0); /* 0 = no esperar si la cola está llena */
    }
}
```

Ctrl+B.

Para explicarlo: `osThreadFlagsWait` duerme la tarea sin gastar CPU hasta que la interrupción la despierta. `osMessageQueuePut` con `timeout 0` hace que, si el filtro va retrasado y la cola está llena, la IMU no se bloquee: esa muestra se pierde.

Paso D: tBaro, con el mutex del I²C

En fTareaBaro, sustituye el `for(;;)`:

```
muestra_t m;
ms5607_data_t baro;
uint32_t siguiente = osKernelGetTickCount();
for (;;)
{
    /* El I2C1 lo comparte con tSalud (INA226): solo uno a la vez */
    if (osMutexAcquire(mtxI2C1Handle, 10) == osOK)
    {
        sens_status_t st = ms5607_read(&baro);
        osMutexRelease(mtxI2C1Handle); /* soltarlo SIEMPRE */
        if (st == SENS_OK)
        {
            m.t_us = osKernelGetTickCount() * 1000U;
            m.sensor = 3; /* 3 = barómetro */
            m.valor[0] = baro.pressure_pa;
            m.valor[1] = baro.temp_c;
            m.valor[2] = 0;
        }
    }
}
```

```

        osMessageQueuePut(qSensoresHandle, &m, 0, 0);
    }
}
siguiente += 20;          /* 20 ms = 50 Hz */
osDelayUntil(siguiente);
}

```

Ctrl+B.

Para explicarlo: el mutex evita que tBaro y tSalud usen el I²C a la vez, y tiene herencia de prioridad contra la inversión de prioridad. osDelayUntil mantiene los 50 Hz exactos, sin acumular deriva.

Limitación honesta: ms5607_read() espera la conversión con HAL_Delay, que con un RTOS bloquea la CPU unos 20 ms. La mejora es cambiarlo por osDelay, que duerme la tarea. Si te lo preguntan, es lo primero que cambiarías.

Paso E: tEstimacion consume la cola

En fTareaEstimacion, sustituye el for(;;):

```

muestra_t m;
estado_t e = {0};
for (;;)
{
    /* Espera la siguiente muestra (de cualquier sensor) */
    if (osMessageQueueGet(qSensoresHandle, &m, NULL, 10) == osOK)
    {
        /* Aquí iría el paso del filtro de Kalman: predicción con la IMU,
        * corrección con barómetro, magnetómetro y GNSS. */
        e.t_us = m.t_us;
        osMessageQueuePut(qEstadoHandle, &e, 0, 0); /* publica la estimación */
    }
}

```

Ctrl+B.

Para explicarlo: osMessageQueueGet duerme la tarea hasta que llega una muestra. Es el patrón productor-consumidor: los sensores producen y el filtro consume.